PIPE-Cypher: Automatic Enterprise Benchmark Generation for Text-to-Cypher Systems

Anonymous Authors

Abstract

Enterprises increasingly use property graphs and Cypher to analyze fraud, risk, identity, customer, and operational data. Public Text2Cypher benchmarks rarely reflect private schemas, terminology, access patterns, or difficulty distributions. We present PIPE-Cypher, a local-model pipeline for generating organization-specific natural-language-to-Cypher benchmarks. PIPE-Cypher combines schema introspection, reverse-query grounding, constrained Cypher generation, deterministic read-only and schema validation, execution feedback, repair, diversity controls, and LLM-judge review. The system is designed for industry settings where benchmark generation must avoid paid APIs, preserve data governance, and remain repeatable as graphs evolve.

1 Introduction

Property graphs encode enterprise relationships directly: account transfers, identity entitlements, access paths, customer interactions, and fraud rings. Cypher is a common query language for these graphs. As LLMs become natural-language interfaces for graph analytics, organizations need reliable benchmarks for natural-language-to-Cypher systems on their own schemas.

Static public benchmarks are insufficient for this setting. They cannot contain private labels, relationship types, categorical values, or domain-specific wording, and they do not track schema evolution. PIPE-Cypher addresses this gap by generating private, repeatable, execution-grounded NL-Cypher benchmarks.

2 Related Work

Recent Text2Cypher resources, including Mind the Query [1], SyntheT2C [9], Auto-Cypher [7], and Text2Cypher [4], show that high-quality Cypher data requires more than asking an LLM for query pairs. These works motivate schema grounding, execution checks, synthetic generation, verification, and complexity-aware evaluation. PIPE-Cypher differs by targeting private enterprise benchmark generation as the primary artifact: organizations bring their own graph, run local models, enforce Cypher constraints, and receive an executable benchmark with diversity and judge metadata.

Text-to-SQL benchmarks such as Spider 2.0 [2] and BIRD [3] motivate realistic database tasks, execution-based evaluation, and enterprise workflow complexity. CIKM AutoQuery [8] motivates strategy-level workload generation analysis and separating workload quality from model quality. LDBC FinBench [6] and SNB [5] provide graph workloads suitable for industry-oriented evaluation.

3 Method

PIPE-Cypher has five stages: schema profiling, template generation, reverse Cypher grounding, constrained Cypher generation and repair, deterministic validation and execution, and LLM-judge review. Determinis-

Gate	Evidence checked	Failure mode caught
Read-only safety	blocked Cypher write/admin tokens	destructive or operational queries
Schema validity	labels, relationship types, properties	hallucinated graph vocabulary
Direction validity	observed relationship directions	reversed or invalid traversals
Cypher normalization	rewrites and `RETURN DISTINCT`	duplicate/noisy rows
Question constraints	quoted values use exact literals	fuzzy matching of exact user values
Execution	query runs and returns rows	syntactic validity without answerability
LLM judge	ambiguity, semantic alignment, schema use	executable but semantically weak examples

Table 1: PIPE-Cypher candidate acceptance gates.

Graph	Target/category	Binding limit	Min seed capacity	All categories meet target
FinBench	250	300	300	Yes
SNB	125	200	200	Yes

Table 2: Built-in seed-template capacity after removing the reverse-binding 10-row cap and adding slotted negation/ranking seeds. Capacity is a theoretical scale-readiness check; final examples still must pass validation, execution, diversity, and judge gates.

tic checks enforce read-only safety, syntax shape, schema-visible labels and properties, relationship types, observed relationship directions, and non-empty execution where required.

4 Implementation

The implementation is a Python package with Neo4j as the experimental backend. The paper frames the contribution around Cypher and property graphs rather than a single vendor. Local models are served through a vLLM/OpenAI-compatible endpoint on ds-serv6, using Qwen3.5 models for generation and judging.

The intended full-quality model is Qwen3.5-35B-A3B, but our June 1, 2026 live evidence uses the cached Qwen3.5-9B fallback. The 35B-A3B weights were staged on ds-serv6 under root-backed storage. A serving-capacity check estimated that the staged weights require four A5000 GPUs under our vLLM budget, while only one GPU was safely free in the live snapshot, so the reported generation and downstream results use the 9B fallback.

The built-in FinBench profile is grounded in the public datagen snapshot export and includes typed node and relationship properties. The generated Neo4j import script uses uniqueness constraints for nodes and creates, rather than merges, transaction relationships so repeated account-to-account events are preserved. The SNB reference profile is grounded in the official Neo4j/Cypher headers and read-query files.

For reproducible smoke runs, PIPE-Cypher can use seeded templates with deterministic reverse-binding queries. Full runs use a mixed mode that prepends these proven workload seeds to LLM-proposed templates, then records accepted and rejected candidates for yield analysis.

For LLM-judge review, the implementation slices schema context to the labels, relationship types, and properties used by the candidate query. This preserves validation against the full schema while keeping local 9B smoke-model prompts within context limits on larger schemas.

We also estimate seed-template capacity before full generation. This check caught and fixed a scale blocker: reverse-binding execution was hard-capped to 10 rows, and no-slot negation/ranking seeds could not support full category targets. PIPE-Cypher now uses the configured binding limit and includes slotted negation and ranking seeds for both graphs.

Setting	Value
Accepted examples	3,000
Primary graph	LDBC FinBench, 2,000 examples
Secondary graph	LDBC SNB, 1,000 examples
Categories	8 balanced categories, 375 each
Generation/judge model	Local Qwen3.5-9B fallback
Execution backend	Neo4j Community, two live databases
Judge audit packet	80 sampled accepted/rejected pairs

Table 3: Full live experimental setup used for the generated benchmark artifact.

Graph	Candidates	Accepted	Acceptance	Categories at target
FinBench	3,404	2,000	0.588	8/8
SNB	1,373	1,000	0.728	8/8
Total	4,777	3,000	0.628	16/16

Table 4: Full live generation with local Qwen3.5-9B. Candidate counts include the initial sequential run and category-specific recovery top-ups.

5 Experiments

The generated benchmark contains 3,000 accepted examples: 2,000 from LDBC FinBench and 1,000 from LDBC SNB. Categories include simple retrieval, complex retrieval, simple aggregation, complex aggregation, boolean existence, negation/difference, path/temporal transaction, and ranking/top-k.

6 Results

The full live run produced 3,000 accepted examples from 4,777 candidates using local Qwen3.5-9B for generation and judging. Category-specific recovery top-ups filled the only under-target categories from the initial sequential run. Every exported example passed read-only, syntax, schema, execution, non-empty result, and judge gates.

As an implementation smoke check, we generated and loaded FinBench SF0.1 into Neo4j, loaded the official SNB Cypher test-data into a second Neo4j instance, introspected both live schemas, served Qwen3.5-9B locally through vLLM, and ran four-category smokes on both graphs. All eight accepted examples passed read-only, syntax, schema, execution, non-empty result, and LLM-judge gates. A broader seeded FinBench smoke additionally accepted 8/8 examples across all planned categories with four easy and four medium examples.

We also ran a small live mini-ablation. A FinBench LLM-only probe accepted 0/16 candidates, with deterministic validation tagging all 16 as generic unlabeled node scans. Re-enabling seeded templates, scalar reverse-binding, duplicate-question control, deterministic fallback, execution validation, and LLM-

Metric	Offline	Live FinBench	Live SNB
Records generated	4	4	4
Accepted records	4	4	4
Read-only pass	4	4	4
Syntax-valid pass	4	4	4
Schema-valid pass	4	4	4
Execution-success pass	4	4	4
Judge pass	4	4	4

Table 5: Smoke evidence. The live runs used loaded Neo4j graphs and local Qwen3.5-9B judge review; these numbers verify end-to-end wiring but are not final experimental results.

Live run	Records	Accepted	Acceptance
FinBench LLM-only probe	16	0	0.000
FinBench mixed mini	29	16	0.552
SNB mixed mini	8	8	1.000

Table 6: Live mini-ablation evidence with local Qwen3.5-9B. The LLM-only probe disables deterministic seed/fallback, while mixed runs combine seeded templates with LLM-proposed templates and full validation/judge gates.

judge review accepted 16/29 FinBench candidates, with two accepted examples in every planned category. The analogous SNB mixed run accepted 8/8 candidates.

We additionally materialized the planned baseline and ablation configurations and ran target-five suites on both FinBench and SNB. The strict unconstrained local-LLM baseline disables seeded template fallback, retrieval, rewrite, repair, deterministic Cypher fallback, and the LLM judge; it produced no usable records on either graph because the local model did not return parseable template JSON. Once reverse grounding and seeded workload structure are available, the small target saturates across variants, which is why the full results emphasize scale, recovery, and export quality rather than only small-yield differences.

We then ran a live mid-scale generation pass with a target of five accepted examples per category for each graph. FinBench accepted 40/46 candidates and SNB accepted 40/47 candidates; both runs reached the category target in all eight planned categories.

The accepted full-run records were exported into a benchmark package with stable example identifiers, train/dev/test splits, result samples, gate metadata, aggregate statistics, and a manifest hash for artifact tracking. The export contains 3,000 accepted examples: 2,000 FinBench, 1,000 SNB, and 375 accepted examples in every planned category.

For judge calibration, the released artifacts include an 80-row audit packet sampled from accepted and rejected full-run candidates. The current draft treats these labels as pending human review rather than as a generation gate.

Setting	Graph	Records	Accepted	Acceptance	Categories at target
Unconstrained LLM	FinBench	0	0	0.000	0/8
Unconstrained LLM	SNB	0	0	0.000	0/8
Reverse-only	FinBench	40	40	1.000	8/8
Reverse-only	SNB	40	40	1.000	8/8
Validators+repair	FinBench	40	40	1.000	8/8
Validators+repair	SNB	40	40	1.000	8/8
No retrieval	FinBench	41	40	0.976	8/8
No retrieval	SNB	40	40	1.000	8/8
No rewrite	FinBench	41	40	0.976	8/8
No rewrite	SNB	40	40	1.000	8/8
No LLM judge	FinBench	40	40	1.000	8/8
No LLM judge	SNB	40	40	1.000	8/8
Full PIPE-Cypher	FinBench	41	40	0.976	8/8
Full PIPE-Cypher	SNB	40	40	1.000	8/8

Table 7: Live target-five ablation evidence with local Qwen3.5-9B. Each graph run targets 5 accepted examples per category.

Live run	Records	Accepted	Acceptance	Categories at target
FinBench mid-scale	46	40	0.870	8/8
SNB mid-scale	47	40	0.851	8/8

Table 8: Live mid-scale generation evidence with local Qwen3.5-9B. Each run targets five accepted examples in each planned category.

Finally, we ran a downstream Text2Cypher evaluation using local Qwen3.5-9B on the exported full test split. The model saw schema text and the natural-language question, generated Cypher, and was evaluated by live execution against the corresponding FinBench or SNB database. The result verifies that the exported artifact can drive downstream model evaluation and gives a difficulty signal: the zero-shot 9B baseline solves simple aggregation examples but struggles on more operational categories.

7 Industry Use

PIPE-Cypher is intended to be rerun when an enterprise graph changes. The generated artifact records the schema snapshot, graph profile, model identifier, validation gates, execution samples, judge scores, difficulty features, and source run for every example. Long-running jobs can be monitored from JSONL records and recovered with category-specific top-up runs that reject questions accepted in earlier runs. This design supports private benchmark refreshes without exposing schemas or values to paid APIs, while still leaving audit hooks for data owners to sample accepted and rejected examples.

Export artifact	Examples	FinBench	SNB	Train	Dev	Test
Live full benchmark	3,000	2,000	1,000	2,408	296	296

Table 9: Accepted live full benchmark package with stable IDs, gate metadata, result samples, statistics, and manifest hash `8bc79a53a06b291a`.

Artifact property	Value
Categories	8 balanced categories
FinBench/category	250
SNB/category	125
Difficulty split	1,521 easy / 1,479 medium
Unique labels / rel. types	7 / 9
Read/syntax/schema/exec/judge gates	3,000/3,000

Table 10: Distribution and gate summary for the exported full benchmark artifact.

8 Limitations and Ethics

Execution validity does not guarantee semantic correctness. LLM judges can over-accept plausible but wrong examples, so the final study includes a small human audit for calibration. Generated artifacts may contain private schema details or sampled values, so organizations should apply normal data governance controls.

9 Conclusion

PIPE-Cypher provides a practical path for enterprises to create private, executable, balanced NL-to-Cypher benchmarks. The pipeline combines schema introspection, constrained generation, deterministic validation, execution feedback, diversity control, and automated judge review.

References

[1] Vashu Chauhan, Shobhit Raj, Shashank Mujumdar, Avirup Saha, and Anannay Jain. Mind the query: A benchmark dataset towards Text2Cypher task. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1890–1905, Suzhou, China, 2025. Association for Computational Linguistics.

[2] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows, 2024.

Split	Examples	Parse	Schema	Exec. success	Exec. acc.	Answer F1
Live full test	296	0.959	0.905	0.622	0.189	0.189

Table 11: Downstream Text2Cypher evaluation for local Qwen3.5-9B on the exported full benchmark test split.

[3] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can LLM already serve as a database interface? a BIg bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems*, 2023.

[4] Makbule Gulcin Ozsoy, Leila Messallem, Jon Besga, and Gianandrea Minneci. Text2Cypher: Bridging natural language and graph databases. In *Proceedings of the Workshop on Generative AI and Knowledge Graphs*, pages 100–108, Abu Dhabi, UAE, 2025. International Committee on Computational Linguistics.

[5] David P
üroja, Jack Waudby, Peter Boncz, and G
'abor Sz
'arnyas. The LDBC social network benchmark interactive workload v2: A transactional graph query benchmark with deep delete operations, 2023.

[6] Shipeng Qi, Heng Lin, Zhihui Guo, G
'abor Sz
'arnyas, Bing Tong, Yan Zhou, Bin Yang, Jiansong Zhang, Zheng Wang, Youren Shen, Changyuan Wang, Parviz Peiravi, Henry Gabb, and Ben Steer. The LDBC financial benchmark, 2023.

[7] Aman Tiwari, Shiva Krishna Reddy Malay, Vikas Yadav, Masoud Hashemi, and Sathwik Tejaswi Madhusudhan. Auto-cypher: Improving LLMs on cypher generation via LLM-supervised generation-verification framework. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2: Short Papers*, pages 623–640, Albuquerque, New Mexico, 2025. Association for Computational Linguistics.

[8] Xiuwen Zheng, Arun Kumar, and Amarnath Gupta. Generating cross-model analytics workloads using LLMs. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pages 4303–4307. ACM, 2024.

[9] Zijie Zhong, Linqing Zhong, Zhaoze Sun, Qingyun Jin, Zengchang Qin, and Xiaofan Zhang. SyntheT2C: Generating synthetic data for fine-tuning large language models on the Text2Cypher task. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 672–692, Abu Dhabi, UAE, 2025. Association for Computational Linguistics.